

Internet Programming I

Chapter 4 JavaScript: Objects

Group: 6

3rd Year Sec D Students,
Dept. of Software Eng.

Jan 2022, AASTU



Objectives



After success completion of the session you will be able to:

- Define what is object
- How to create object in different ways
- Differentiate what is property and method
- Define what is constructor and prototype
- List and discuss built-in objects in javascript
- Discuss how to traverse objects in javascript
- Define what is inheritance

Javascript object and inheritance

- JS Object concept
- Object Creation options
- Object properties and methods
- Object accessors
- Object constructors and prototype
- Traverse objects
- Inheritance

Object concept



- object can be a variable, a data structure, a function, or a method. As regions of memory, they contain value and are referenced by identifiers.
- In the object-oriented programming paradigm, object can be a combination of variables, functions, and data structures; in particular in class-based variations of the paradigm it refers to a particular instance of a class.
- In the relational model of database management, an object can be a table or column, or an association between data and a database entity (such as relating a person's age to a specific person).
- An important distinction in programming languages is the difference between an object-oriented language and an object-based language. A language is usually considered object-based if it includes the basic capabilities for an object: identity, properties, and attributes. A language is considered object-oriented if it is object-based and also has the capability of polymorphism, inheritance, encapsulation, and, possibly, composition.
- JavaScript is a prototype based object oriented language

Javascript Object concept



Objects, in JavaScript, is it's most important data-type and forms the building blocks for modern JavaScript. These objects are quite different from JavaScript's primitive data-types(Number, String, Boolean, null, undefined and symbol) . in the sense that while these primitive data-types all store a single value each (depending on their types).

- Objects are more complex and each object may contain any combination of these primitive data-types as well as reference data-types.
- An object, is a reference data type. Variables that are assigned a reference value are given a reference or a pointer to that value. That reference or pointer points to the location in memory where the object is stored. The variables don't actually store the value.

Javascript Object concept



- Loosely speaking, objects in JavaScript may be defined as an unordered collection of related data, of primitive or reference types, in the form of “key: value” pairs. These keys can be variables or functions and are called properties and methods, respectively, in the context of an object.

Javascript Object creating



1. Creating objects using object literal syntax

This is really simple. All you have to do is throw your key value pairs separated by ':' inside a set of curly braces({ }) and your object is ready to be served (or consumed), like below:

```
const person = {  
  firstName: 'testFirstName',  
  lastName: 'testLastName'  
};
```

This is the simplest and most popular way to create objects in JavaScript.

Javascript Object creating



2. Creating objects using the 'new' keyword

This method of object creation resembles the way objects are created in class-based languages, like Java.

a) Using the 'new' keyword with in-built Object constructor function

To create an object, use the new keyword with `Object()` constructor, like this:

```
const person = new Object();
```

Now, to add properties to this object, we have to do something like this:

```
person.firstName = 'testFirstName';
```

```
person.lastName = 'testLastName';
```

Javascript Object creating



b) Using 'new' with user defined constructor function

problem with the approach of using the 'Object' constructor function result from the fact that every time we create an object, we have to manually add the properties to the created object. so we use user defined constructor.

```
function Person(fname, lname) {  
    this.firstName = fname;  
    this.lastName = lname;  
}
```

```
const personOne = new Person('testFirstNameOne', 'testLastNameOne');
```

```
const personTwo = new Person('testFirstNameTwo', 'testLastNameTwo');
```

Javascript Object creating



3. Using Object.create() to create new objects

The `Object.create()` method creates a new object using the prototype of the given object.

```
let Student = {
  name: "Lisa",
  age: 24,
  marks: 78.9,
  display() {
    console.log("Name:", this.name);
  }
};

// create object from Student prototype
let std1 = Object.create(Student);

std1.name = "Sheeran";
std1.display();

// Output: Name: Sheeran
```

Javascript Object creating



create() Syntax

The syntax of the `create()` method is:

```
const orgObject = { company: 'ABC Corp' };
```

And you want to create employees for this organization. Clearly, you want all the employee objects.

```
const employee = Object.create(orgObject, { name: { value:  
'EmployeeOne' } });
```

```
console.log(employee); // { company: "ABC Corp" }
```

```
console.log(employee.name); // "EmployeeOne"
```

4.Object.assign()

`Object.assign` method can take any number of objects as parameters. The first parameter is the object that it will create and return. The rest of the objects passed to it will be used to copy the properties into the new object. Let's understand it by extending the previous example we saw.

- `Object.assign()` is used for cloning an object.
- `Object.assign()` is used to merge object with same properties.

Javascript Object creating



Assume you have two objects as below:

```
const orgObject = { company: 'ABC Corp' }
```

```
const carObject = { carName: 'Ford' }
```

Now, you want an employee object of 'ABC Corp' who drives a 'Ford' car. You can do that with the help of `Object.assign` as below:

```
const employee = Object.assign({}, orgObject, carObject);
```

Now, you get an employee object that has `company` and `carName` as its property.

```
console.log(employee); // { carName: "Ford", company: "ABC Corp" }
```

Object properties and methods

Object Properties

In JavaScript, an object is a collection of properties, where each property is a key-value pair. This Example creates a new object called `person` using an object initializer.

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};
```

The `person` object has two properties `firstName` and `lastName`.

Object properties and methods

➤ Manipulating Properties of an Object

Properties can usually be changed, added, and deleted, but some are read only.

A. Accessing Properties

To access the properties of the object we can use

1. **Dot property accessor** `expression.identifier`

expression should evaluate to an object, and identifier is the name of the property you'd like to access.

`person.firstName`

expression:

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};  
console.log(person.firstName)  
console.log(person.lastName)
```

Object properties and methods cont...

You can use the dot property accessor in a chain to access deeper properties: `object.prop1.prop2`.

Choose the dot property accessor when the property name is *known ahead of time*.

The dot property accessor works correctly when the property name is a valid identifier.

```
const person = {  
  'first-Name': 'John',  
  'last Name': 'Doe',  
  '3': 3  
};  
console.log(person.first-Name)  
console.log(person.'last Name')  
console.log(person.3)
```

An identifier in JavaScript contains Unicode letters, \$, _, and digits 0..9, but cannot start with a digit.

Object properties and methods cont...

2. Square brackets property accessor ([])

The square brackets property accessor has the following syntax:

```
expression[expression]
```

The first expression should evaluate to an object and the second expression should evaluate to a string denoting the property name.

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};  
console.log(person[firstName])  
console.log(person[lastName])
```

when a property name contains spaces, you need to place it inside quotes

```
let address = {  
  'building no': 3960,  
  street: 'Churchill Ave',  
  state: 'Addis Ababa',  
  country: 'Ethiopia'  
}  
console.log(address['building no'])
```

Object properties and methods cont...

To access the properties with these special names, use the square brackets property accessor

```
const person = {  
  'first-Name': 'John',  
  'last Name': 'Doe',  
  '3': 3,  
};  
console.log(person['first-Name'])  
console.log(person['last Name'])  
console.log(person[3])
```

The square brackets syntax accesses without problems the properties that have special names

3. Object destructuring



Reading Assignment

The basic object destructuring syntax is pretty simple:

<https://dmitripavlutin.com/access-object-properties-javascript/>

```
const { identifier } = expression;
```

`identifier` is the name of the property to access and `expression` should evaluate to an object.

Object properties and methods cont...

B. Change the Property's value

To change the value of a property, you use the assignment operator

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};  
  
person.firstName = 'Josi';  
console.log(person);
```

Output:

```
{ firstName: 'Josi', lastName: 'Doe' }
```

C. Add a new property to an object

Unlike objects in other programming languages such as Java and C#, you can add property to an object after creating it

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};  
  
person.age = '26';  
console.log(person);
```

Output:

```
{ firstName: 'John', lastName: 'Doe', age: '26' }
```

Object properties and methods cont...

D. Delete a property of an object

To delete a property from an object, you use the `delete` operator

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};  
  
person.age = '26';  
delete person.age;  
console.log(person);
```

Output:

```
{ firstName: 'John', lastName: 'Doe' }
```

E. Check if a property exists

To check if a property exists in an object, you use the `in` operator

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};  
  
console.log('age' in person);  
console.log('firstName' in person);
```

Output:

```
false  
true
```

Object properties and methods cont...

Object Methods

Objects have actions. The actions are represented by functions. the following snippet adds the greet action to the person object

```
const person = {
  firstName: 'John',
  lastName: 'Doe'
};
const greet = function(){
  console.log('Hello, World!')
}

person.greet = greet()
```

```
const person = {
  firstName: 'John',
  lastName: 'Doe'
};
person.greet = function(){
  console.log('Hello, World!')
}

person.greet()
```

```
const person = {
  firstName: 'John',
  lastName: 'Doe',
  greet : function(){
    console.log('Hello, World!')
  }
};
person.greet()
```

Object properties and methods cont...

The `this` value

Typically, methods need to access data stored in the object.

For example you want to develop a method that returns the full name of the person object by concatenating the first name and last name.

Inside the method, the `this` value references the object that contains the methods so you can access an object property using the dot notation

```
const person = {
  firstName: 'John',
  lastName: 'Doe',
  greet : function(){
    console.log('Hello, World!')
  },
  getFullName: function(){
    return this.firstName+" "+this.lastName;
  }
};
console.log(person.getFullName());
```

Output:

```
John Doe
```

Object Property types

Javascript specifies characteristics of properties of objects via internal attributes surrounded by the two pair of square brackets eg, `[[Enumerable]]`

There are two types of object properties **data properties** and **accessor properties**

Data Properties

A data property contains a single location for a data value. A data property has four attributes:

- `[[Configurable]]`- determines whether a property can be redefined or removed via `delete` operator
- `[[Enumerable]]`-indicates that if a property will be returned in the `for...in` loop;
- `[[Writable]]`-specifies that the value of a property can be changed
- `[[Value]]`-contains the actual value of a property

Object properties and methods cont...

Data Properties

By default, the `[[Configurable]]`, `[[Enumerable]]` and `[[Writable]]` attributes set to true for all properties defined directly on an object. The default value of `[[Value]]` attribute is undefined.

To change any attribute of a property you use the `Object.defineProperty()` method.

`Object.defineProperty()` method accepts three arguments:

- An object
- A property name of the object
- A property descriptor object that has four properties:
 - `Configurable`
 - `Enumerable`
 - `Writable`
 - `Value`

Object properties and methods cont...

Data Properties

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe',  
  age: 26,  
};  
Object.defineProperty(person, 'age', {  
  configurable: false,  
})  
delete person.age;  
console.log(person.age);
```

Output

26

Output

30

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe',  
  age: 26,  
};  
Object.defineProperty(person, 'age', {  
  configurable: false,  
  enumerable: false,  
  value: 30,  
  writable: false,  
})  
delete person.age;  
person.age = 34;  
console.log(person.age);
```

Object properties and methods cont...

Accessor Properties

similar to data properties, accessor properties also have `[[Configurable]]` and `[[Enumerable]]` attributes. But the accessor properties have the `[[Get]]` and `[[Set]]` attributes instead of `[[Value]]` and `[[Writable]]`.

When you read data from an accessor property, the `[[Get]]` function is called to return a value. The default return value of the `[[Get]]` function is undefined .

If you assign a value to an accessor property, the `[[set]]` function is called. To define an accessor property, you must use the `Object.defineProperty()` method

Object properties and methods cont...

Accessor Properties

```
const person = {
  firstName: 'John',
  lastName: 'Doe',
  age: 26,
};
Object.defineProperty(person, 'fullName', {
  get: function(){
    return this.firstName+' '+this.lastName
  },
  set: function(value){
    let part = value.split(' ')
    this.firstName = part[0];
    this.lastName = part[1];
  }
})
person.fullName = "Zerubabel Kassahun";
console.log(person.fullName);
console.log(person.firstName);
console.log(person.lastName);
```

Output:

```
Zerubabel Kassahun
Zerubabel
Kassahun
```

Object accessors

```
const person = {  
  firstName: 'John',  
  lastName: 'Doe',  
  get fullName(){  
    return person.firstName + " " + person.lastName;  
  },  
  set fullName(value){  
    const part = value.split(' ');  
    this.firstName = part[0];  
    this.lastName = part[1];  
  }  
}  
  
console.log(person.fullName);  
person.fullName = 'Samuel Noah';  
console.log(person.fullName);  
console.log(person.firstName)  
console.log(person.lastName)
```

Output:

```
John Doe  
Samuel Noah  
Samuel  
Noah
```

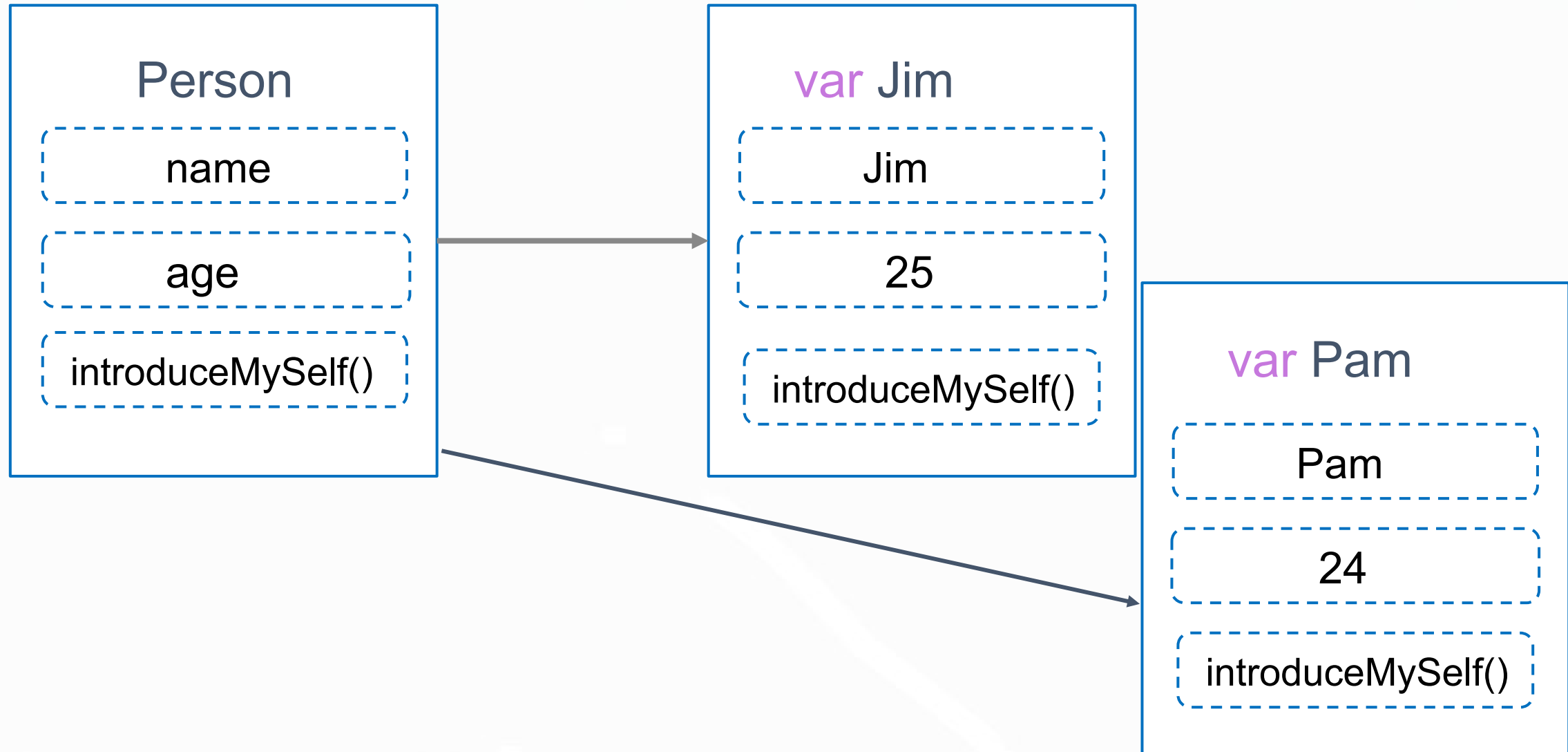
Object constructors and Prototype



➤ Object Constructor

- Instead of creating individual objects, what if we wanted to create several instance objects from a single “*Blueprint*” object? This is when constructor functions come into play.
- **Constructor** - is a function used to create instance objects with the **new** keyword.
- When you call a constructor, it will:
 - create a new object
 - bind `this` to the new object, so you can refer to `this` in your constructor code
 - run the code in the constructor
 - return the new object.

Object constructors and Prototype



Object constructors and Prototype



```
1
2 //Creating a constructor
3 const Person = function(name, age){
4     this.name = name;
5     this.age = age;
6     this.introduceMySelf = _=>{
7         return `I am ${this.name}. and I am ${this.age} years old.`;
8     }
9 }
10
11 //Creating an instance object
12 const abebe = new Person('Abebe', 24);
13 console.log(abebe.introduceMySelf());
14
```

PROBLEMS 5 OUTPUT TERMINAL DEBUG CONSOLE

```
zadicus@DESKTOP-GTQB3BG MINGW64 ~/Desktop/OOP
$ node testRun.js
I am Abebe. and I am 24 years old.
```

- ❖ We are creating an “abebe” object from the constructor “Person” having the attributes “name”, “age” and a method “intruduceMySelf”.
- ❖ Can you think of another way of creating an instance object from an Object?

Object constructors and Prototype Cont...



- ❖ The “**new**” keyword is used to create an instance of an object that has a constructor function. On calling the constructor function with ‘**new**’ operator, the following actions are taken:
 - A new empty object is created.
 - The new object’s internal ‘Prototype’ property (`__proto__`) is set the same as the prototype of the constructing function.
 - The ‘this’ variable is made to point to the newly created object. It binds the property which is declared with ‘this’ keyword to the new object.
 - A newly created object is returned when the constructor function returns a non-primitive value (custom JavaScript object). If the constructor function returns a primitive value, it will be ignored. At the end of the function, ‘this’ is returned if there is no return statement in the function body.

- ❖ Can anyone explain the “**this**” keyword function?

➤ Prototype

- JavaScript is a prototype based language, so, whenever we create a function using JavaScript, JavaScript engine adds a *prototype* property inside a function, **Prototype property** is basically an object (also known as Prototype object), where we can attach methods and properties in a prototype object, which enables all the other objects to inherit these methods and properties.
- When we create an object using function constructor, JavaScript Engine will add dunder proto or `__proto__` in the object which will point to the prototype's constructor object.

Object constructors and Prototype Cont...



Output

```
// function constructor
const Person = function(name, job, yearOfBirth){

    this.name= name;
    this.job= job;
    this.yearOfBirth= yearOfBirth;
}

Person.prototype.calculateAge= function(){
    console.log('The current age is: '
    +(2019- this.yearOfBirth));
}

console.log(Person.prototype);
```

```
▼ Object i
  ▼ calculateAge: f ()
    arguments: null
    caller: null
    length: 0
    name: ""
  ► prototype: {constructor: f}
    [[FunctionLocation]]: VM35:8
  ► [[Prototype]]: f ()
  ► [[Scopes]]: Scopes[2]
  ► constructor: f (name, job, yearOfBirth)
  ► [[Prototype]]: Object
```

Practical Exercises



1. What do you think the output for the following piece of code and Why?:

```
// function constructor
const Person = function(name){
    this.name= name;
}
Person.prototype.nationality = "Russian";
const John = new Person("John");
console.log(John.__proto__.nationality);
const Abera = new Person("Abera");
Abera.__proto__.nationality = "Ethiopian";
console.log(Abera.__proto__.nationality);
console.log(John.__proto__.nationality);
```

TASK 1:

- Create a web page with two buttons "TakeOff" and "Land", and an empty text field area.
- Write an Airplane constructor that initializes `name` from an argument.
- All airplanes built with Airplane should initialize with an `isFlying` of false.
- Give airplanes the ability to `.takeOff()` and `.land()`:
 - + If a plane takes off, its `isFlying` property is set to true.
 - + If a plane lands, its `isFlying` property is set to false.
- When the takeOff button is clicked, "IsFlying = True" should be rendered to the empty text area in the web page created.
- When the Land button is clicked, "IsFlying = False" should be rendered to the empty text area in the web page created.

Solution for Task 1

```
<!doctype html>
<html>
<head><title>Lab Exercies</title></head>
<body>
<button id="takeOffBtn">TakeOff</button>
<button id="landBtn">Land</button>
<div id="textField"></div>
<script src="./app.js"></script>
</body>
</html>
```



The image shows a UI mockup for a landing simulation. It features a rounded rectangular container. At the top, there are two buttons: "TakeOff" on the left and "Land" on the right. Below these buttons is a large rectangular text area, which is currently empty and labeled "Text Area" in the center.

Solution for Task 1

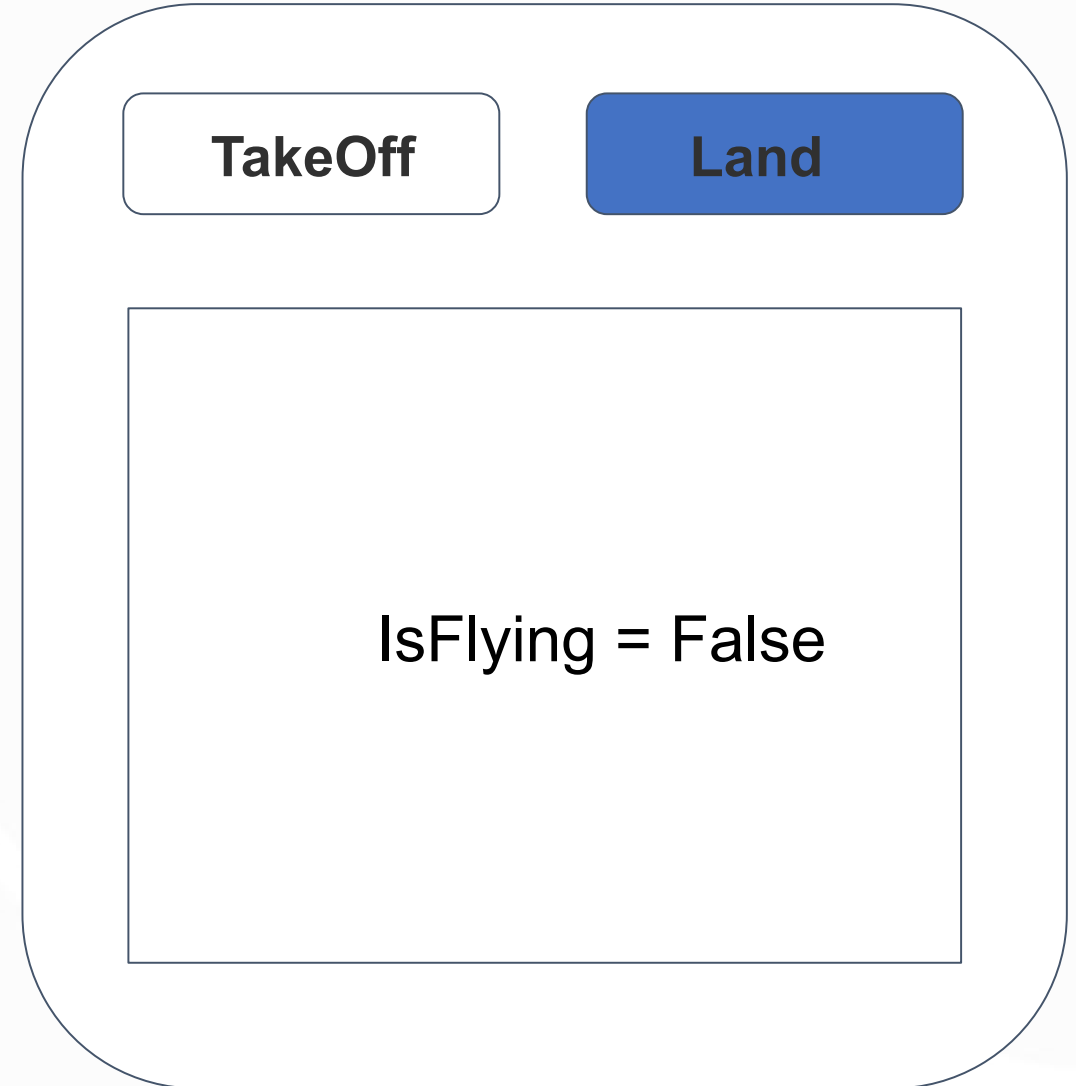
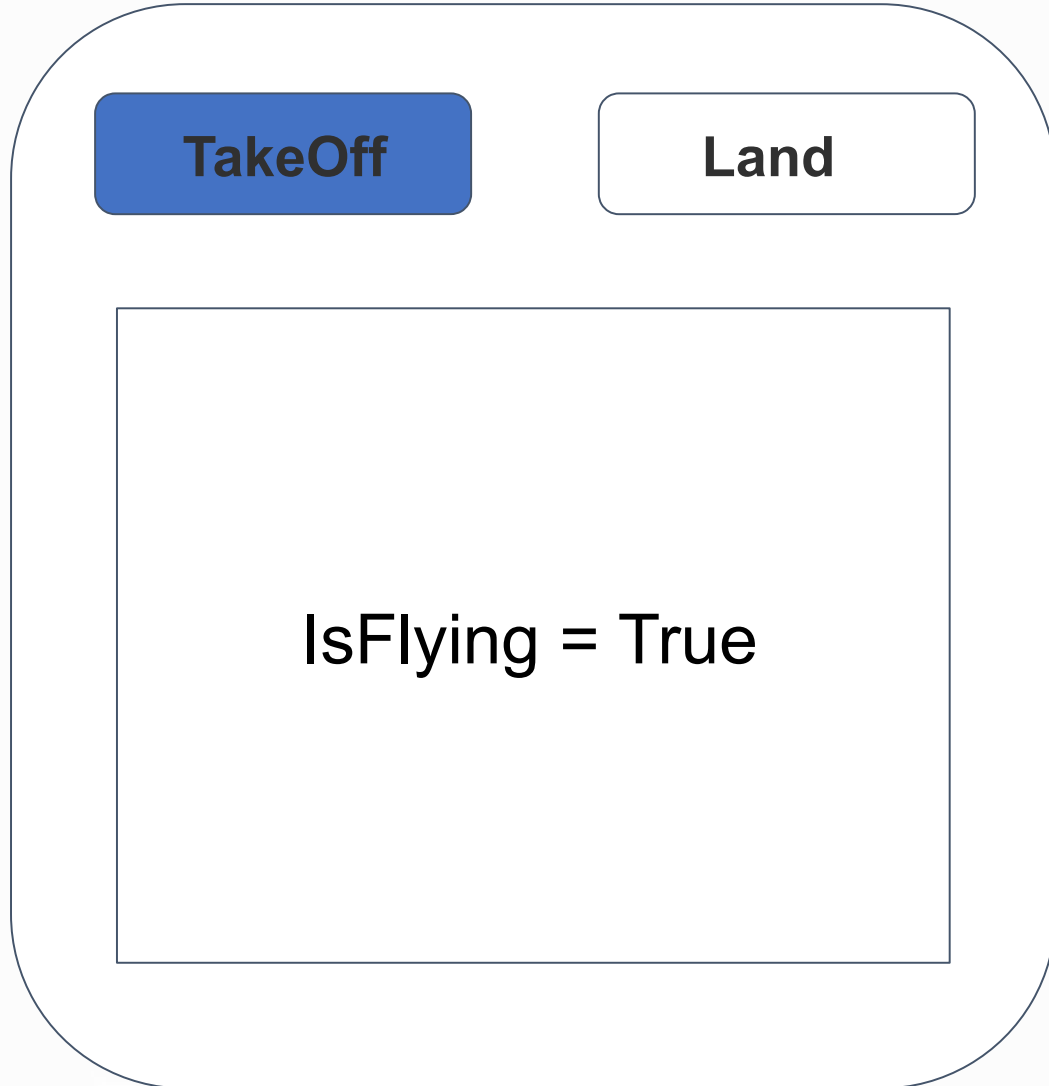
```
function Airplane(name) {  
    this.name = name;  
    this.isFlying = false;  
}  
Airplane.prototype.takeOff = function() {  
    this.isFlying = true;  
};  
Airplane.prototype.land = function() {  
    this.isFlying = false;  
};  
Airplane boing = new Airplane("Boing");
```

Solution for Task 1

```
const takeOffBtn = document.querySelector("#takeOffBtn");
takeOffBtn.onclick = function() {
    boing.takeOff();
    document.querySelector("#textArea").innerText = `IsFlying = $
{boing.isFlying}`;
}

const landBtn = document.querySelector("#landBtn");
landBtn.onclick = function() {
    boing.takeOff();
    document.querySelector("#textArea").innerText = `IsFlying = $
{boing.isFlying}`;
}
```

Solution for Task 1



TASK 2

- Write a Person Constructor that initializes ``name`` and ``age`` from arguments.
- All instances of Person should initialize with an empty ``stomach`` array.
- Give instances of Person the ability to ``.eat("someFood")``:
 - + When eating an edible, it should be pushed into the ``stomach``.
 - + The ``eat`` method should have no effect if there are 10 items in the ``stomach``.
- Give instances of Person the ability to ``.poop()``:
 - + When an instance poops, its ``stomach`` should empty.
- Give instances of Person a method ``.toString()``:
 - + It should return a string with ``name`` and ``age``. Example: "Mary, 50"

Solution for Task 2

```
function Person(name, age) {  
    this.name = name;  
    this.age = age;  
    this.stomach = [];  
}  
  
Person.prototype.eat = function(food) {  
    if (this.stomach.length < 10) {  
        return this.stomach.push(food);  
    } else {  
        return `I can't eat anymore`;  
    }  
};  
  
Person.prototype.poop = function() {  
    return (this.stomach = []);  
};  
  
Person.prototype.toString = function() {  
    return `${this.name}, ${this.age}`;  
};
```

TASK 3

- Create a web page with a form or you can use the register page from activity 4.
- Write a javascript program that creates an object and sets its values from the web page form input.
- Render the object back to the web page.

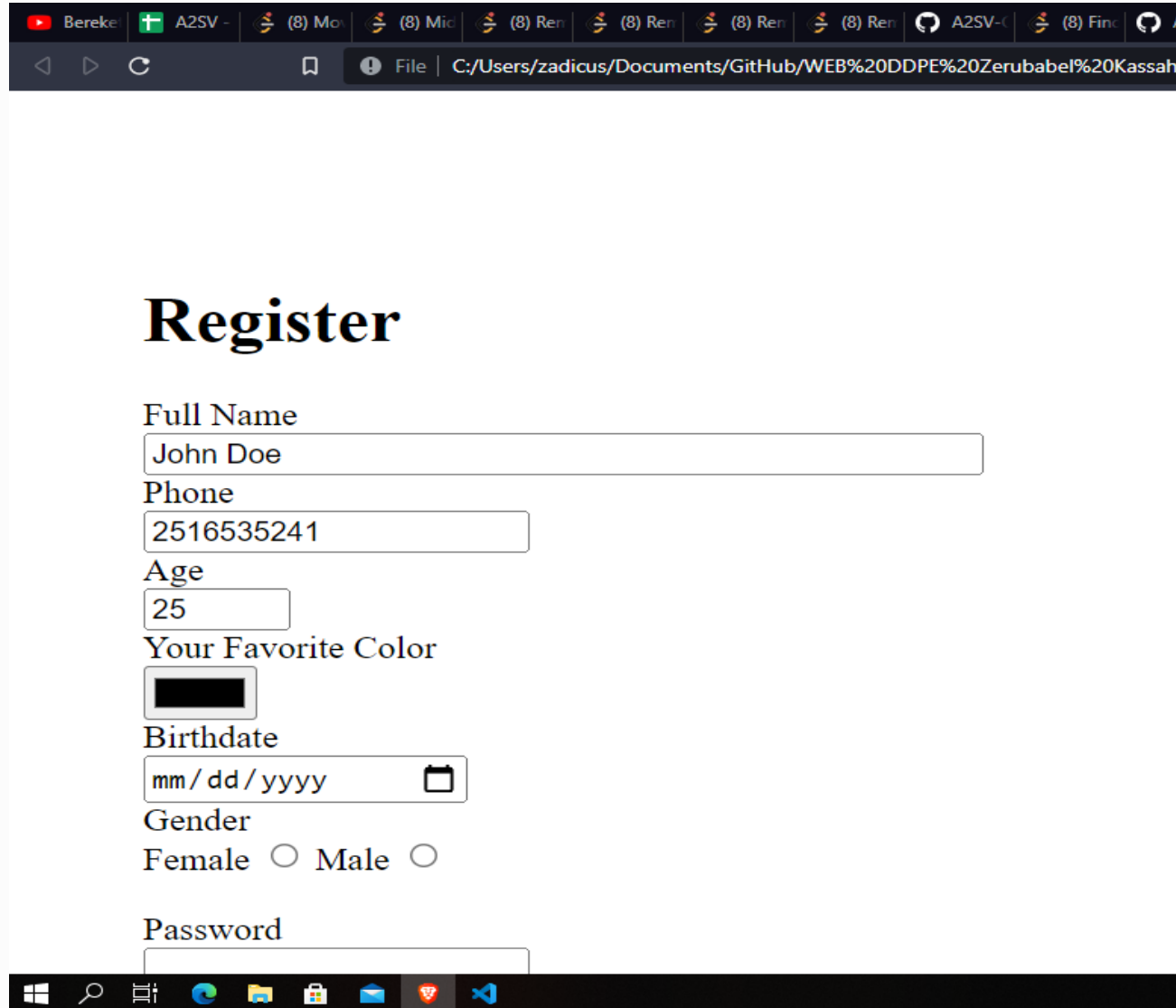
Solution

```
function Register() {
  this.fullName = document.getElementById('fullname').value;
  this.phone = document.getElementById('phone').value;
  this.age = document.getElementById('age').value;
  this.introduceMySelf = () => {
    parent = document.getElementById('render');
    const div = document.createElement('div');
    div.classList.add('appended');
    div.innerHTML = `<h1>New Added Form</h1><h1>Full Name: ${this.fullName}</h1><h1>Age: $
{this.age}</h1><h1>PhoneNo.: ${this.phone}</h1>`;
    parent.appendChild(div)
  }
}

const callBack = () => {
  const newObject = new Register();
  newObject.introduceMySelf()
}

document.getElementById('submitBtn').addEventListener('click', callBack);
```

Solution

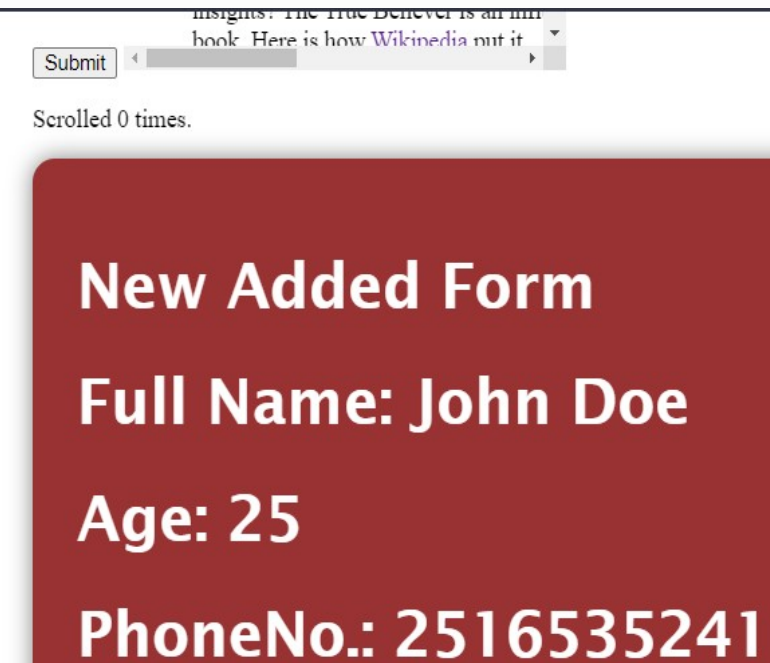


The screenshot shows a web browser window with a dark theme. The address bar displays the file path: C:/Users/zadicus/Documents/GitHub/WEB%20DDPE%20Zerubabel%20Kassahu. The page content includes a registration form with the following fields and values:

- Full Name:** John Doe
- Phone:** 2516535241
- Age:** 25
- Your Favorite Color:** A black color selection box.
- Birthdate:** mm/dd/yyyy (with a calendar icon)
- Gender:** Female ☐ Male ☐
- Password:** (empty field)

The Windows taskbar is visible at the bottom of the screen.

Output



The output shows the result of the form submission. It includes a 'Submit' button, a 'Scrolled 0 times.' message, and a large red box containing the following text:

- New Added Form**
- Full Name: John Doe**
- Age: 25**
- PhoneNo.: 2516535241**

Below the red box, there is a link to follow the project on social media: Follow Us On: [Facebook](#) | [Twitter](#).

Inheritance and Prototype chain

- When it comes to inheritance, JavaScript only has one construct: objects. Each object has a private property which holds a link to another object called its **prototype**. That prototype object has a prototype of its own, and so on until an object is reached with `null` as its prototype. By definition, `null` has no prototype, and acts as the final link in this **prototype chain**.

```
function Person(firstName, lastName)
{
  this.FirstName = firstName || "unknown";
  this.LastName = lastName || "unknown";
};
Person.prototype.getFullName = function ()
{ return this.FirstName + " " +
  this.LastName; }
```

- ❖ Now any objects instantiated from the “Person” constructor will inherit the properties and methods inside the “Person” object.
- ❖ All objects are an instance of the “Object” object. This is the last object in the prototype chain before the `Null` prototype object. Therefore, all objects created in javascript inherit the methods and properties pre-defined in the “Object” object.

Practical Exercises

1. Can an inherited method from the “Object” class be modified?
1. What do you think the output of the following code will be:

```
function Dog(name) {  
    this.name = name;  
    this.speak = function() {  
        return 'woof';  
    };  
}  
  
const dog = new Dog('Pogo');  
Dog.prototype.speak = function() {  
    return 'arf';};  
console.log(dog.speak());
```

1. Traverse objects

➤ what it mean traverse objects?

- looping through objects .
- The various methods that can be used to loop through objects in JavaScript are
 - Using a for...in loop
 - Object.keys method
 - Object.values method

1. Traverse objects

➤ Methods to loop through objects using javascript

1.Using a for...in loop

The most straightforward way to loop through an object's properties is by using the for...in statement

```
function person (name,age) {  
  this.name=name;  
  this.age=age;  
  
}  
  
var b=new person("Abebe",12);  
  for (const key in b) {  // iterate over the user object  
    console.log(`${key}: ${b[key]}`);  
  }  
  
// name: Abebe  
// age: 12
```

1. Traverse objects

- ❑ One problem in using the for...in method is that it loops through the properties in the prototype chain as well. Since the objects in JavaScript can inherit properties from their prototypes, the for...in statement will loop through those properties as well. but to avoid this problem, you have to explicitly check if the property belongs to the object by using the **hasOwnProperty()** method. example:

```
function person (name,age){this.name=name;
this.age=age;}
var b=new person("Abebe",12);
person.prototype.bloodColor="red";
for (var key in b) {
  //if(person.hasOwnProperty(key);
  console.log(`${key}: ${b[key]}`);
} // name: Abebe // age: 12 //bloodColor:red
```

1. Traverse objects

➤ Methods to loop through objects using javascript

1.Object.keys() Method :-

It takes the object that you want to loop over as an argument and returns an array containing all properties names (or keys)

```
function person (name,age) {  
  this.name=name;  
  this.age=age;  
}  
var Abebe=new person("Abebe",12);  
const keys = Object.keys(Abebe);  
person.prototype.bloodColor="red";  
  
  keys.forEach(function(index) {  
    console.log(`${index}: ${Abebe[index]}`);  
  });  
// output name: Abebe age: 12
```

1. Traverse objects

➤ Methods to loop through objects using javascript

1.Object.values() Method :-

It takes the object that you want to loop over as an argument and returns an array containing all properties names (or keys)

```
function person (name,age) {  
  this.name=name;  
  this.age=age;  
}  
  
var Abebe = new person("Abebe", 12);  
  Object.values(Abebe).forEach(function (val) {  
console.log(val) });
```

```
// output  name: Abebe  age: 12
```

Reading Resources/Materials

Links

- ✓ <https://www.geeksforgeeks.org/>
- ✓ <https://developer.mozilla.org/en-US/>
- ✓ <https://www.w3schools.com/>

Thank You
For Your Attention!!

Any Questions

